

به نام خدا

زبان‌های برنامه‌نویسی

آرش شفیعی



- مفاهیم زبان‌های برنامه‌نویسی، از روبرت سبستا¹
- مفاهیم در زبان‌های برنامه‌نویسی، از جان میچل²
- زبان‌های برنامه‌نویسی : اصول و الگوواره‌ها، از ماریتسیو گابریلی³
- مستندات زبان‌های برنامه‌نویسی

¹ Concepts of Programming Languages, by Robert W. Sebesta

² Concepts in Programming Languages, by John C. Mitchell

³ Programming Languages : Principles and Paradigms, by Maurizio Gabbrielli

مقدمه

- یک زبان برنامه‌نویسی، یک سیستم نشانه‌گذاری¹ است برای بیان یک برنامه کامپیوتری². یک برنامه کامپیوتری مجموعه‌ای از دستورات است برای انجام محاسبات بر روی داده‌ها.
- یک زبان برنامه‌نویسی ایده‌آل، به برنامه‌نویسان اجازه می‌دهد یک برنامه را به طور مختصر و واضح بیان کنند.
- از آنجایی که یک برنامه کامپیوتری باید در طول دوران حیات خود توسط برنامه‌نویسان مختلف خوانده شود، فهمیده شود، و تغییر داده شود، بنابراین یک زبان برنامه‌نویسی خوب زبانی است که برنامه‌نویسان را قادر می‌سازد، برنامه‌های یکدیگر را بهتر بفهمند.
- معمولاً یک برنامه بزرگ از تعداد زیادی اجزا تشکیل شده است که با یکدیگر در ارتباط هستند. یک زبان برنامه‌نویسی خوب، زبانی است که به برنامه‌نویسان کمک می‌کند برنامه‌های پیچیده و ارتباط بین اجزا را به سادگی و به طور بهینه توصیف کنند.

¹ Notation system

² Computer program

- در طراحی یک زبان برنامه‌نویسی معمولاً باید انتخاب‌های مختلف را سبک‌سنگین¹ کرد.
- به طور مثال برخی از ویژگی‌های زبان‌های برنامه‌نویسی به برنامه‌نویسان کمک می‌کنند برنامه‌ها را سریع بنویسند، اما از طرفی همین ویژگی‌ها باعث کاهش بهره‌وری² و پیچیده‌تر شدن و کندتر شدن کامپایلر می‌شوند.
- برخی از ویژگی‌های زبانی باعث می‌شوند کامپایلر به طور بهینه پیاده‌سازی شود، اما از طرف دیگر باعث می‌شوند برنامه برای برنامه‌نویسان پیچیده‌تر شود.
- از آنجایی که برنامه‌های متفاوت نیازهای متفاوتی دارند، بنابراین زبان‌های برنامه‌نویسی متفاوتی به وجود آمده‌اند تا این نیازها را برآورده سازند. هر زبان برنامه‌نویسی موفق در ابتدا برای یک مجموعه از برنامه‌ها با نیازهای مشابه به وجود آمده است. البته این بدین معنی نیست که یک زبان فقط برای انجام یک کار به وجود آمده است، بلکه بدین معنی است که تمرکز یک زبان بر روی حل کردن مجموعه‌ی مشخصی از مسائل است.

¹ Trade-off

² Performance

- مطالعه زبان‌های برنامه‌نویسی به ما کمک خواهد کرد مفاهیم اصلی در زبان‌های برنامه‌نویسی را بهتر بفهمیم و با نقاط قوت و ضعف آنها بهتر آشنا شویم و بتوانیم از زبان‌ها حداکثر بهره را ببریم.
- هر زبان برنامه‌نویسی روشی متفاوت برای حل یک مسئله ارائه می‌کند. در واقع یک زبان برنامه‌نویسی یک چهارچوب فکری برای حل مسئله و طراحی نرم‌افزار ارائه می‌کند. بنابراین مطالعه یک زبان و روش‌های موجود در آن زبان جهت حل یک مسئله، می‌تواند به حل آن مسئله در زبان‌های دیگر نیز کمک کند.

- در اینجا ابتدا توضیح می‌دهیم چرا به مطالعه زبان‌های برنامه‌نویسی احتیاج داریم.
- سپس به طور خلاصه حوزه‌هایی را توضیح می‌دهیم که در آنها به زبان‌های برنامه‌نویسی نیاز داریم.
- توضیح می‌دهیم بر اساس چه معیارهایی می‌توان زبان‌های برنامه‌نویسی را مورد ارزیابی قرار داد.
- در پایان تاریخچه زبان‌های برنامه‌نویسی و روش‌های پیاده‌سازی آنها را به اختصار شرح می‌دهیم.

- افزایش توانایی بیان ایده‌ها : معمولا عمق تفکر افراد وابسته به توان بیان آنها در زبانی است که با استفاده از آن ارتباط برقرار می‌کنند. از آنجایی که زبان ابزاری است برای تفکر و اندیشیدن، هر چقدر یک زبان را بهتر بشناسیم، بهتر می‌توانیم با استفاده از آن ایده‌ها را منتقل و مسائل را حل کنیم. معمولا یک زبان ساختار معینی دارد و با استفاده از آن ساختار می‌توان ارتباط برقرار کرد ولی هر ساختاری محدودیت‌هایی نیز دارد. بنابراین یادگیری یک زبان متفاوت که ساختار متفاوتی دارد کمک می‌کند بتوانیم بهتر بیاندیشیم.

دلایل مطالعه زبان‌های برنامه نویسی

- برنامه نویسان نیز در طراحی نرم‌افزار محدود به زبانی هستند که با استفاده از آن برنامه می‌نویسند. معمولاً در برنامه نویسی ساختارهای کنترلی و ساختارهای داده موجود بر روش تفکر ما در حل یک مسئله محدودیت اعمال می‌کنند. یادگیری زبان‌های متفاوت کمک می‌کند که ساختارهای متفاوتی در ذهن ما به وجود بیایند و بتوانیم برای حل مسئله از آن ساختارها استفاده کنیم.
- برای مثال یک برنامه نویس سی که با پایتون آشنا باشد و از سهولت استفاده از لیست‌ها آگاه باشد، می‌تواند لیست‌هایی مشابه با پایتون در برنامه سی ایجاد کند و از آنها برای حل مسئله خود استفاده کند.

دلایل مطالعه زبان‌های برنامه نویسی

- انتخاب مناسب یک زبان برای یک کاربرد خاص: معمولاً یک زبان برای کاربردی خاص به وجود می‌آید و ممکن است کسی که در یک حوزه فعالیت می‌کند و زبانی را برای کاربردی خاص استفاده می‌کند، از زبان‌هایی که برای کاربردهای دیگر وجود دارند بی‌اطلاع باشد. مطالعه زبان‌های برنامه‌نویسی کمک می‌کند نقاط قوت و ضعف و کاربرد زبان‌ها را به خوبی بشناسیم و هر زبان را مناسب با حوزه مرتبط با آن به کار ببریم.
- افزایش توانایی در یادگیری زبان‌های جدید: علوم کامپیوتر و تکنولوژی‌های مرتبط با آن همواره در حال پیشرفت هستند و زبان‌های جدید در حال ابداع شدن هستند. یادگیری مفاهیم اصلی در طراحی زبان‌های برنامه نویسی به ما کمک می‌کند تا بتوانیم زبان‌های جدید را بهتر یاد بگیریم. برای مثال وقتی با مفهوم برنامه نویسی شیء گرا آشنا شدیم می‌توانیم از شیء گرایی در همه‌ی زبان‌های شیء گرا استفاده کنیم. این اصل در یادگیری زبان‌های طبیعی نیز صادق است. هرچه با گرامرهای بیشتری در زبان‌های متفاوت آشنا باشیم یادگیری یک زبان طبیعی جدید برای ما آسان‌تر می‌شود.

دلایل مطالعه زبان‌های برنامه نویسی

- استفاده بهتر از یک زبان: وقتی با نحوه پیاده سازی یک زبان آشنا شویم، می‌توانیم از آن به طور بهینه‌تر استفاده کنیم. به طور مثال وقتی بفهمیم در زبان‌های شیء گرا، وراثت چه سر باری تحمیل می‌کند متوجه می‌شویم چه جاهایی بهتر است از وراثت استفاده کنیم و چه جاهایی از آن استفاده نکنیم. به عنوان مثالی دیگر، برنامه نویسی که از سربار و پیچیدگی فراخوانی توابع آگاه نباشد، ممکن است از توابع به کثرت استفاده کند که این امر باعث کاهش بهره‌وری برنامه می‌شود. بنابراین با یادگیری مفاهیم پایه در طراحی زبان‌ها می‌توانیم برنامه نویس بهتری شویم. همچنین یادگیری مفاهیم برنامه نویسی به یک برنامه نویس کمک می‌کند تا از قابلیت‌هایی که تاکنون استفاده نکرده است، استفاده کند.
- ابداع زبان‌های جدید: در نهایت با مطالعه مفاهیم زبان‌های برنامه نویسی می‌توانیم دیدگاه روشن‌تری نسبت به گستره‌ی زبان‌های برنامه نویسی به دست آوریم و بنابراین می‌توانیم زبان‌های جدیدی را به فراخور نیازهای خود ابداع و پیاده سازی کنیم.

- کامپیوترها و برنامه‌های کامپیوتر در حوزه‌های مختلفی استفاده می‌شوند. از سیستم‌های تعبیه شده در هواپیما گرفته تا سیستم‌های محاسباتی پیچیده در آزمایشگاه‌های تحقیقاتی تا بازی‌های کامپیوتری در کامپیوترهای خانگی یا تلفن‌های همراه.

- کاربردهای علمی: اولین کامپیوترهای دیجیتال در دهه ۱۹۴۰ و ۱۹۵۰ برای انجام محاسبات علمی و استفاده در آزمایشگاه‌های تحقیقاتی به وجود آمدند. در آن زمان برنامه‌های پیچیده و ساختار داده‌های پیچیده وجود نداشتند، اما به محاسبات پیچیده نیاز بود. معمولاً برای انجام چنین محاسباتی به آرایه و ماتریس و یک حلقه برای شمارش نیاز بود. بنابراین زبان‌هایی که در آن زمان ابداع شدند، در جهت رفع نیازها طراحی شده بودند. قبل از آن دوره از زبان اسمبلی استفاده می‌شد. اولین زبانی که برای محاسبه فرمول‌های ریاضی به وجود آمد زبان فورترن^۱ بود. در آن دوره زبان الگول^۲ نیز به وجود آمد. اما زبان فورترن به نحوی برای آن کاربرد خاص طراحی شده بود که هنوز هم مورد استفاده است. زبان‌های آن زمان به دلیل محدودیت سخت‌افزار به کارایی قابل توجهی نیاز داشتند.

^۱ Fortran

^۲ Algol

- **کاربردهای تجاری:** استفاده از کامپیوترها برای کاربردهای تجاری در ابتدای دهه‌ی ۱۹۵۰ آغاز شد. اولین زبانی که در این حوزه به وجود آمد زبان کوپول^۱ بود. در کاربردهای تجاری نیاز به سهولت در گزارش‌گیری و همچنین انجام دقیق محاسبات تجاری است. توسعه برنامه‌های کاربردی برای انجام محاسبات در بانک‌ها یکی از موارد کاربردهای تجاری است.
- **هوش مصنوعی:** یکی از کاربردهای مهم در محاسبات کامپیوتر در حوزه هوش مصنوعی است. در این حوزه از ماتریس‌ها و آرایه‌ها و انجام محاسبات بر روی این ساختارها به کثرت استفاده می‌شود. اولین زبانی که در این حوزه به وجود آمد زبان لیسپ^۲ بود که یک زبان تابعی است و در سال ۱۹۵۹ ابداع شد. در دهه ۱۹۷۰ زبان دیگری برای کاربردهای هوش مصنوعی به نام زبان پرولوگ^۳ به وجود آمد. در دهه اخیر بسیاری از برنامه‌های هوش مصنوعی با استفاده از پایتون^۴ نوشته می‌شوند.

^۱ Cobol

^۲ Lisp

^۳ Prolog

^۴ Python

- نرم افزارهای وب: صفحه‌های ساده وب در بدو ابداع آن با استفاده از زبان اچ تی ام ال¹ طراحی می شدند. با پیشرفت تکنولوژی وب و نیاز به صفحه های پویا (صفحه‌هایی که محتوای آن قابل تغییر است)، نیاز به زبان‌های دیگری شد که می‌توان در این دسته به جاوا اسکریپت² یا پی‌اچ‌پی³ اشاره کرد.

¹ HTML

² JavaScript

³ PHP

- برای ارزیابی زبان‌های برنامه نویسی تعدادی معیار را در نظر می‌گیریم.

- (۱) خوانایی : یکی از مهم‌ترین معیارها برای ارزیابی زبان‌های برنامه نویسی، خوانایی است، یعنی سهولت خواندن و فهمیدن برنامه‌ای که توسط برنامه نویسان دیگر نوشته شده است. قبل از ۱۹۷۰ به علت محدودیت‌های سخت‌افزار، آنچه در یک برنامه بیشتر اهمیت داشت کارایی یک برنامه بود. به تدریج برنامه‌های پیچیده‌تری به وجود آمدند و همچنین سخت‌افزارهای قوی‌تر ابداع شد، بنابراین به مرور زمان آنچه اهمیت بیشتری می‌یافت، خوانایی برنامه‌ها بود. همچنین نگهداری برنامه‌ها با به وجود آمدن برنامه‌های پیچیده و بزرگ اهمیت بیشتری پیدا می‌کرد، بنابراین لازم بود برنامه‌ها به میزان کافی خوانا باشند. پس زبان‌های برنامه نویسی که تا آن زمان به زبان ماشین شباهت بیشتری داشتند، به مرور زمان به زبان انسان شباهت بیشتری پیدا کردند. خوانایی را می‌توان از جنبه‌های مختلفی بررسی کرد : سادگی،¹ تعامل¹، نوع‌های داده‌ای، طراحی نحوی².

¹ Orthogonality

² Syntax design

- **سادگی:** سادگی یک زبان برنامه نویسی، بر خوانایی آن تأثیر مستقیم دارد. زبانی که تعداد زیادی ساختار کنترلی و کلمات کلیدی دارد طبیعتاً برای یادگیری سخت‌تر است. وقتی یک زبان، بسیار پیچیده می‌شود، معمولاً برنامه نویسان تنها از قسمتی از زبان استفاده می‌کنند.
- این سادگی باید در حد متوسط باشد تا برنامه بهترین خوانایی را داشته باشد. اگر یک برنامه بیش از حد ساده باشد خوانایی آن کاهش می‌یابد. برای مثال، زبان اسمبلی زبان بسیار ساده‌ای است اما خواندن یک برنامه در زبان اسمبلی نسبت به زبان سی سخت‌تر است.

- **تعامد:** تعامد در یک زبان برنامه نویسی بدین معناست که مجموعه‌ای از ساختارهای ابتدایی در یک زبان بتوانند به چندین روش محدود با یکدیگر ترکیب شوند و همه ساختارهای داده و کنترلی مورد نیاز را بسازند. اگر همه ساختارهای ابتدایی بتوانند با یکدیگر ترکیب شوند و ترکیب‌های معناداری بسازند، می‌گوییم یک زبان تعامد بالایی دارد. تعامد کم در یک زبان بدین معناست که برخی ترکیب‌ها معنادار نیستند. بنابراین یک زبان با تعامد کم استثنای بیشتری دارد. تعداد استثنای زیاد در یک زبان یادگیری زبان را پیچیده تر می‌کند.
- برای مثال در کامپیوترهای IBM دو دستور برای جمع وجود داشت. دستور `Reg, Memory A` محتوای یک رجیستر و یک خانه حافظه را جمع می‌کند و دستور `Reg1, Reg2 AR` محتوای دو رجیستر را جمع می‌کند. بنابر این برخی از ترکیب‌ها در این زبان بی‌معنا هستند. مثلاً توسط دستور `A` نمی‌توان دو رجیستر را با هم جمع کرد که این یک استثنا در زبان است. اما در کامپیوترهای سری `VAX` تنها یک دستور `ADDL` `op1, op2` برای جمع طراحی شده بود. بنابراین رجیسترها و حافظه‌ها به شکل‌های مختلف می‌توانند با استفاده از این دستور با هم جمع شوند و در نتیجه زبان این نوع کامپیوترها تعامد بیشتری دارد.

- اگر تعامد کم باشد، تعداد استثناها افزایش می‌یابد و در نتیجه نوشتن و خواندن برنامه سخت می‌شود. از طرفی دیگر اگر تعامد زیاد باشد، ترکیب‌های پیچیده‌ای توسط زبان به وجود می‌آیند و باز هم خوانایی برنامه کاهش می‌یابد.
- در زبان سی برای مثال یک ساختمان (استراکت) را می‌توانیم توسط یک تابع بازگردانیم ولی یک آرایه را نمی‌توانیم بازگردانیم. اعضای یک ساختمان می‌توانند از هر نوعی باشند غیر از void. مقادیر به توابع با مقدار داده می‌شوند اما اگر آرایه به تابع ارسال کنیم، با ارجاع به تابع داده می‌شود. نوع `long long int` وجود دارد ولی `long long double` وجود ندارد. این استثناها از تأثیرات تعامد کم است که زبان را برای یادگیری و استفاده پیچیده تر می‌کند.
- از طرفی اگر تعامد خیلی زیاد باشد نیز زبان پیچیده می‌شود. مثلاً در زبان الگول نوع‌ها می‌توانند با هم ترکیب شوند و نوع‌های بسیار پیچیده‌ای بوجود آورند. در این حالت هیچ استثنایی وجود ندارد و بنابراین تعامد بسیار بالا است ولی همین امر خوانایی برنامه را کاهش می‌دهد.

- **نوع داده:** وجود امکانات کافی برای نوع داده‌ها و ساختمان داده‌ها در یک زبان به خوانایی آن زبان کمک می‌کند. برای مثال در زبانی که نوع منطقی وجود ندارد، مجبوریم برای تعریف مقادیر درست و نادرست از اعداد صفر و یک استفاده کنیم که این امر موجب کاهش خوانایی کد می‌شود.
- **طراحی قواعد نحوی:** قواعد نحوی در یک زبان تأثیر مهمی بر روی خوانایی برنامه در آن زبان دارد. برای مثال در زبان فورترن برای اتمام یک بلوک از کد از یک کلمه کلیدی متناسب با آن بلوک استفاده می‌شود و بدین ترتیب خوانایی کد در آن نسبت به زبان سی که همه بلوک‌ها در آن با آکولاد پایان می‌یابند بالاتر است. مثلاً برای خاتمه بلوک IF در فورترن از IF END استفاده می‌شود. در اینجا می‌بینیم که برای بالا بردن خوانایی در یک زبان می‌توان کلمات کلیدی را افزایش داد ولی از طرفی افزایش کلمات کلیدی، زبان را پیچیده تر و برای یادگیری سخت تر می‌کند، بنابراین رعایت حد اعتدال در اینجا نیز بسیار دارای اهمیت است.

- همچنین برای خوانایی بهتر، قواعد نحوی باید به گونه‌ای طراحی شوند که صورت و معنی یک عبارت همخوانی داشته باشند. برای مثال در زبان سی اگر متغیری درون یک تابع با استفاده از کلمه کلیدی static تعریف شود، آن متغیر در اولین اجرای تابع بر روی حافظه در بخش داده تعریف می‌شود و مقدار خود را در فراخوانی‌های پی در پی حفظ می‌کند. اما اگر متغیری به صورت عمومی با کلمه static تعریف شود، آن متغیر فقط در آن فایل قابل استفاده است. پس این کلمه کلیدی معانی متعددی دارد که این امر از خوانایی برنامه می‌کاهد ولی از طرف دیگر تعداد کلمات کلیدی کم باعث سادگی زبان می‌شود.

- (۲) قابلیت اطمینان^۱: یک برنامه قابل اطمینان است اگر همیشه همان کاری را انجام دهد که برنامه‌نویس انتظار دارد. به عبارت دیگر برای افزایش قابلیت اطمینان خطاهای برنامه نویس باید توسط کامپایلر تشخیص داده شود.
- از جمله عواملی که بر قابلیت اطمینان تأثیر می‌گذارند، عبارتند از بررسی نوع^۲، مدیریت استثنا^۳، نام‌های مستعار^۴، و خوانایی

^۱ Reliability

^۲ Type checking

^۳ Exception handling

^۴ Aliasing

- بررسی نوع: بررسی نوع بدین معناست که خطاهای نوع داده‌ای در یک برنامه توسط کامپایلر یا در هنگام اجرا تشخیص داده شوند. بررسی نوع در زمان اجرا هزینه بالایی دارد بنابراین در زبانی که به سرعت اجرای زیاد نیاز دارد، بررسی نوع توسط کامپایلر انجام می‌شود. همچنین اگر خطاها در زمان کامپایل شناخته شوند، می‌توان آنها را زودتر رفع کرد و نیازی به اجرای برنامه برای رفع خطا نیست. برای مثال در نسخه‌های اولیه زبان سی، هنگام ارسال یک متغیر به تابع، لزومی نداشت نوع آن متغیر با نوع داده‌ای که تابع دریافت می‌کند همخوانی داشته باشد. این امر موجب خطاهای احتمالی توسط برنامه نویس می‌شد و قابلیت اطمینان زبان را پایین می‌آورد.

- مدیریت استثنا: توانایی یک برنامه برای تشخیص خطاها در زمان اجرا و ادامه برنامه در صورت بروز خطا قابلیت اطمینان برنامه را بالا می‌برد. به این قابلیت مدیریت استثنا گفته می‌شود. برای مثال وجود مدیریت استثنا در زبان سی++ این زبان را نسبت به سی قابل اطمینان‌تر می‌کند.
- نام‌های مستعار: با استفاده از قابلیت ایجاد نام‌های مستعار، یک خانه حافظه می‌تواند دو یا چند نام داشته باشد. برای مثال اشاره‌گرها و متغیرهای ارجاعی در سی++ می‌توانند به یک متغیر چندین نام را منتسب کنند. وجود چنین قابلیت‌ای امکان بروز خطا را بیشتر و در نتیجه قابلیت اطمینان را کمتر می‌کند.
- خوانایی: هرچه خوانایی یک زبان بیشتر باشد، برنامه نویس راحت‌تر می‌تواند برنامه‌هایی بنویسد که درست‌تر و در نتیجه قابل اطمینان‌تر باشند.

- (۳) هزینه: انگیزه ابتدایی طراحی زبان‌های برنامه نویسی پایین آوردن هزینه نوشتن آنها بود. برنامه نویسان می‌توانستند با استفاده از یک زبان برنامه نویسی سطح بالا یک برنامه را در زمان کمتری بنویسند. حال هر چه یک زبان ساده‌تر باشد یادگیری آن نیز نیازمند زمان کمتری است و نوشتن برنامه توسط آن ساده‌تر است و در نتیجه هزینه مالی کمتری برای آن صرف می‌شود.
- هزینه زمانی اجرای یک برنامه نیز یکی از معیارهای ارزیابی یک زبان است. هرچه یک زبان در زمان کمتری یک برنامه معین را اجرا کند، آن برنامه بهتر است. اما گاهی این معیار با معیارهای دیگر در تضاد است. برای مثال اشاره‌گرها گرچه قابلیت اطمینان را پایین می‌آورند، اما از هزینه اجرا نیز می‌کاهند و بهبود می‌دهند.

- گاهی (برای مثال در یک برنامهٔ حسابداری) خوانایی یک برنامه برای ما مهم‌تر است پس زبانی را انتخاب می‌کنیم که خواناتر باشد و هزینه اجرا برای ما در اولویت نیست. اما گاهی (برای مثال در سیستم‌های تعبیه شده در هواپیما) زمان اجرا بسیار پر اهمیت است و خوانایی اهمیت زیادی ندارد.
- هزینه می‌تواند با قابلیت اطمینان به گونه‌ای دیگر نیز در ارتباط باشد. قابلیت اطمینان پایین ممکن است باعث بروز خطا شود و در یک سیستم حساس مانند هواپیما، ممکن است باعث تحمیل هزینه‌های مالی زیادی شود.
- هزینه نگهداری یک نرم‌افزار نیز معیار مهمی است. هرچه خوانایی یک برنامه بالاتر باشد، هزینه نگهداری آن پایین‌تر است چرا که برنامه نویسان آتی می‌توانند برنامه را به آسانی فرا بگیرند و تغییر دهند.

- در بسیاری مواقع یک زبان برنامه‌نویسی قسمتی از وظیفه برنامه‌نویس را به طور خودکار انجام می‌دهد. به طور مثال یک برنامه‌نویس، وظیفه دارد حافظه را مدیریت کند، اما برخی از زبان‌های برنامه‌نویسی مدیریت حافظه را به طور اتوماتیک و خودکار انجام می‌دهند و حافظه‌های تخصیص داده شده را وقتی به آنها دیگر نیازی نیست به طور خودکار آزادسازی می‌کنند. گرچه چنین خودکارسازی‌هایی باعث می‌شود برنامه‌نویس نیازی به فکر کردن نداشته باشد، اما از طرف دیگر باعث می‌شود از سرعت اجرای برنامه‌ها در یک زبان برنامه‌نویسی کاسته شود. وجود ساختارها و نوع‌های داده‌ای متنوع قدرت بیان را بیشتر می‌کند.
- قدرت بیان در مقابل کارایی¹: هر چه قدرت بیان یک زبان بیشتر باشد (کارهای بیشتری توسط زبان به طور خودکار انجام شوند)، برنامه‌نویس نیاز به زمان کمتری برای نوشتن برنامه دارد، اما برنامه با سرعت کمتری اجرا می‌شود و کارایی کمتری دارد. برخی مواقع نیاز است برنامه قدرت بیان بیشتری داشته باشد تا پیچیدگی برنامه کمتر شود و خطاهای برنامه کاهش یابد و برخی مواقع نیاز است که یک برنامه کارایی بیشتری داشته باشد.

¹ Expressiveness versus efficiency

- معیارهای دیگر: یکی از معیارهای مقایسه زبان‌ها می‌تواند قابلیت اجرای برنامه‌های آن بر روی سیستم‌ها و معماری‌های سخت‌افزاری مختلف یا قابلیت جابجایی¹ باشد. هرچه یک زبان دارای استانداردهای بهتری باشد، کامپایلرهای متنوع در سیستم عامل‌های مختلف همگون‌تر پیاده‌سازی می‌شوند و بنابراین یک کد واحد را می‌توان بر روی سیستم‌های مختلف کامپایل و اجرا کرد. به عنوان یک معیار دیگر می‌توانیم عمومی بودن یا اختصاصی بودن یک زبان را در نظر بگیریم. برخی از زبان‌ها برای یک استفاده خاص به کار می‌روند و برخی از زبان‌ها در کاربردهای متنوع‌تری می‌توانند استفاده شوند.
- در طراحی زبان‌های برنامه نویسی معمولاً معیارهای زیادی وجود دارد که این معیارها با هم در تضادند و بدست آوردن حد وسط مناسب برای یک کاربرد خاص بسیار حائز اهمیت است. برای مثال در زبان جاوا بررسی می‌شود که دسترسی به اندیس‌های آرایه در محدوده تعریف شده آرایه باشد. بدین ترتیب قابلیت اطمینان جاوا از سی بیشتر است ولی این بررسی اجرای برنامه جاوا را کندتر می‌کند، پس هزینه اجرا افزایش پیدا می‌کند.

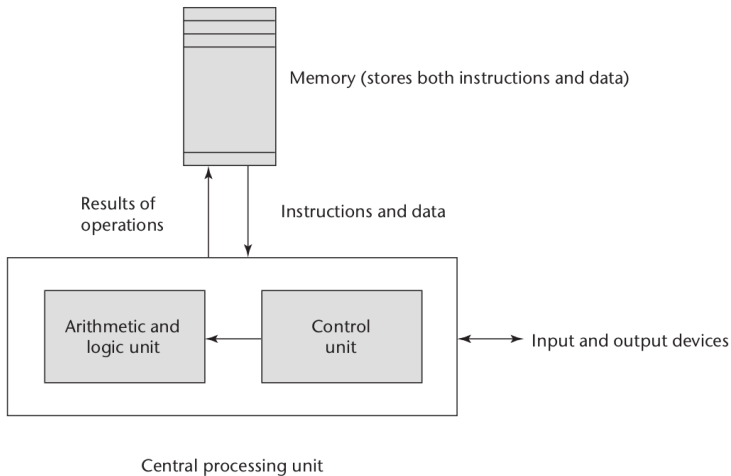
¹ Portability

عوامل تأثیر گذار بر طراحی زبان

- دو عامل مهم تأثیر گذار بر طراحی زبان‌های برنامه‌نویسی عبارتند از معماری کامپیوتر و متوذهای طراحی.
- معماری کامپیوتر: معماری سخت‌افزار عامل تأثیرگذار مهمی در طراحی زبان‌های برنامه‌نویسی است. همه زبان‌های ۶۰ سال گذشته تحت تأثیر معماری وان نویمان^۱ بوده‌اند. در معماری وان نویمان کد برنامه و داده‌ها هر دو بر روی حافظه اصلی قرار می‌گیرند. سپس واحد پردازنده مرکزی، دستورات برنامه را یک به یک از حافظه می‌خواند و اجرا می‌کند. نتیجه محاسبات دوباره بر روی حافظه قرار می‌گیرد. همه کامپیوترها از دهه ۱۹۴۰ تا کنون بر اساس این معماری ساخته شده‌اند که این معماری نیز بر اساس ماشین تورینگ طراحی شده و ماشین تورینگ نیز از موتور تحلیلی چارلز بابج الهام گرفته است. بنابراین همه زبان‌ها نیز طبیعتاً تحت تأثیر این معماری بوده‌اند.

¹ Van Neumann

عوامل تأثیر گذار بر طراحی زبان



شکل: معماری وان نویمان

عوامل تأثیر گذار بر طراحی زبان

- به دلیل استفاده از این معماری اکثر زبان‌های برنامه نویسی از متغیرها استفاده می‌کنند که معرف خانه‌های حافظه است، از عملیات انتساب استفاده می‌کنند که همان عملیات تغییر مقادیر توسط پردازنده است و از حلقه‌ها استفاده می‌کنند که ساده‌ترین روش برای تکرار دسته‌ای از دستورات است.

عوامل تأثیر گذار بر طراحی زبان

- اجرای یک برنامه بر روی این معماری بدین شکل است که پردازنده دستورات را از حافظه می‌خواند و یک به یک اجرا می‌کند. آدرس آخرین دستور برای اجرا در یک رجیستر به نام رجیستر شمارنده برنامه¹ ذخیره می‌شود.

- می‌توان الگوریتم اجرای برنامه را به صورت زیر نوشت :

```
۱ initialize the program counter
۲ repeat forever
۳     fetch the instruction pointed to by the program counter
۴     increment the program counter to point at the next instruction
۵     decode the instruction
۶     execute the instruction
۷ end repeat
```

¹ program counter

عوامل تأثیر گذار بر طراحی زبان

- برخی از زبان‌های برنامه نویسی عملیات انتساب متغیر ندارند. این زبان‌ها که زبان‌های تابعی نامیده می‌شوند خروجی را براساس اعمال تعدادی توابع بر ورودی محاسبه می‌کنند. این زبان‌ها به طور مستقیم از معماری وان نویمان پیروی نمی‌کنند، اما کامپایلری که برای آنها نوشته می‌شود باید نهایتاً برنامه را به زبان ماشین که براساس معماری وان نویمان است ترجمه کند.
- **متودهای طراحی:** در اوایل دهه ۱۹۷۰ نرم‌افزارها برای برنامه‌های بزرگتری به کار می‌رفتند و بنابراین نیاز به سازماندهی بهتر نرم‌افزارها بود. این امر سبب شد روش‌ها و متودهایی برای طراحی برنامه‌های پیچیده ابداع شود که این متودها منجر به ابداع زبان‌هایی شد که تمرکزشان بر روی ساخت نوع داده‌ها و روش‌های سازماندهی برنامه بود. اولین زبانی که برای حل چنین مشکلاتی به وجود آمد زبان سیمولا^۱ بود که شروعی برای طراحی زبان‌های شیء‌گرا بود. پس از آن زبان‌های اسمالتاک^۲، سی++ و جاوا به وجود آمدند.

^۱ Simula

^۲ Smalltalk

تاریخچه زبان‌های برنامه‌نویسی

- در طول نیم قرن اخیر صدها زبان برنامه‌نویسی طراحی و پیاده‌سازی شده‌اند. بسیاری از زبان‌های برنامه‌نویسی از مفاهیم مشابهی استفاده می‌کنند، بنابراین در اینجا بر روی تعدادی از آنها تمرکز می‌کنیم که مفاهیم متفاوتی را ارائه می‌کنند.
- هر زبان برنامه‌نویسی بر اساس تعدادی الگوواره¹ ساخته شده است.
- یک الگوواره یا پارادایم به طور کلی یک چهارچوب فکری یا مجموعه‌ای از مفروضات و مفاهیم و ارزش‌هاست که الگوهای مشابهی را می‌سازند.
- یک پارادایم یا الگوواره برنامه‌نویسی مجموعه‌ای از ساختارهای برنامه‌نویسی برای دسته‌ای از زبان‌های برنامه‌نویسی است که الگوهای مشابه و ویژگی‌های همسانی را می‌سازند. به عبارت دیگر یک پارادایم ویژگی‌های اصلی دسته‌ای از زبان‌های برنامه‌نویسی را تعیین می‌کند.

¹ paradigm

- یکی از سؤالاتی که همیشه ذهن منطق‌دانان را مشغول کرده بوده است این بوده که آیا می‌توان یک مدل محاسباتی ارائه کرد که هر چه قابل محاسبه است را محاسبه کند؟ در دهه ۱۹۴۰ دو مدل توسط آلن تورینگ و آلونزو چرچ ارائه شد. حساب لامبدا توسط چرچ و ماشین تورینگ توسط دانشجوی چرچ یعنی تورینگ ارائه شد و بعدها هر دو اثبات کردند که این دو مدل از نظر قدرت محاسبات برابرند. با این وجود کامپیوترهای امروزی بر اساس ماشین تورینگ طراحی شده‌اند و نه حساب لامبدا. با این حال دو دسته زبان‌های برنامه‌نویسی به وجود آمد. دسته زبان‌های رویه‌ای بر اساس ماشین تورینگ و دسته زبان‌های تابعی بر اساس مدل چرچ یعنی حساب لامبدا شکل گرفتند.

- دو دسته مهم از روش‌های برنامه‌نویسی که بر اساس دو الگوواره و دو چارچوب فکری به وجود آمده‌اند، عبارتند از: برنامه‌نویسی دستوری²، و برنامه‌نویسی اعلانی³.
- در برنامه‌نویسی دستوری، مراحل اجرای یک برنامه گام به گام توسط برنامه‌نویس بیان می‌شود، در حالی که در برنامه‌نویسی اعلانی تنها هدف انجام محاسبات بدون شرح چگونگی انجام آن توصیف می‌شود.

² imperative programming

³ declarative programming

تاریخچه زبان‌های برنامه‌نویسی

- برای مثال در زبان پایتون¹، به روش برنامه‌نویسی دستوری برای محاسبه عدد فیبوناچی n ام برنامه‌ای به صورت زیر می‌نویسیم:

```
۱ def fib(n) :  
۲     i,j = 0,1  
۳     for k in range(1,n + 1):  
۴         i,j = j, i + j  
۵     return j
```

- همین برنامه در زبان هسکل²، به روش برنامه‌نویسی اعلانی به صورت زیر نوشته می‌شود:

```
۱ fib 0 = 0  
۲ fib 1 = 1  
۳ fib n = fib (n-1) + fib (n-2)
```

¹ Python

² Haskell

- دو پارادایم (الگوارۀ) مهم را می‌توان زیرمجموعهٔ پارادایم برنامه‌نویسی دستوری، به حساب آورد: برنامه‌نویسی رویه‌ای¹ و برنامه‌نویسی شیء‌گرا².

¹ Procedural programming

² Object-oriented programming

تاریخچه زبان‌های برنامه‌نویسی

- در برنامه‌نویسی رویه‌ای، یک برنامه از تعدادی رویه تشکیل شده است. هر رویه مقادیری را به عنوان ورودی می‌گیرد و پس از انجام محاسبات مقادیری را باز می‌گرداند. یک رویه می‌تواند در محاسبات خود از تعدادی متغیر عمومی استفاده کند. زبان‌های مهم در این دسته عبارتند از: فورترن¹، الگول²، کوبول³ و سی.
- در برنامه‌نویسی شیء‌گرا، یک برنامه از تعدادی از اشیاء که با یکدیگر در ارتباط هستند تشکیل شده است. هر شیء نمونه‌ای از یک کلاس است و یک کلاس ویژگی‌ها و رفتارهای معینی دارد. زبان‌های مهم در این دسته عبارتند از: سیمولا⁴، اسمالتاک⁵، سی++، و جاوا.

¹ Fortran

² Algol

³ Cobol

⁴ Simula

⁵ Smalltalk

- دو پارادایم مهم را می‌توان زیرمجموعهٔ پارادایم برنامه‌نویسی اعلانی، به حساب آورد: برنامه‌نویسی تابعی¹ و برنامه‌نویسی منطقی².

¹ Functional programming

² Logic programming

- در برنامه‌نویسی تابعی، یک برنامه از تعدادی تابع تشکیل شده است. هر تابع مقادیری را به عنوان ورودی می‌گیرد و مقادیری را باز می‌گرداند. یک تابع به ازای یک ورودی معین همیشه خروجی ثابتی را باز می‌گرداند. زبان‌های مهم در این دسته عبارتند از: لیسپ¹، ام‌ال²، اوکمل³، هسکل⁴.
- در برنامه‌نویسی منطقی، یک برنامه از عبارات منطقی تشکیل شده است. زبان پرولوگ⁵ در این دسته قرار دارد.

¹ Lisp

² ML

³ Ocaml

⁴ Haskell

⁵ Prolog

- یک پارادایم مهم دیگر در زبان‌های برنامه‌نویسی همروند¹ و توزیع‌شده² است. در برنامه‌نویسی همروند و توزیع‌شده محاسبات به صورت موازی توسط تعدادی پردازنده انجام می‌شود. زبان‌های مهم در این دسته عبارتند از: گو¹، و ارلنگ².

¹ Concurrent programming

² Distributed programming

¹ Go

² Erlang

تاریخچه زبان‌های برنامه‌نویسی

- برخی از زبان‌ها تعدادی از پارادایم‌ها را پشتیبانی می‌کنند. برای مثال زبان پایتون هم یک زبان رویه‌ای است، هم شیء‌گرا، و هم تابعی. همچنین کتابخانه‌ای برای برنامه‌نویسی همروند ارائه می‌کند، بنابراین می‌توان برای برنامه‌نویسی همروند هم از آن استفاده کرد.
- به طور مشابه جاوا و سی++ گرچه در گروه زبان‌های شیء‌گرا قرار می‌گیرند، اما کتابخانه‌هایی را ارائه می‌کنند که با استفاده از آنها می‌توان به صورت تابعی و همروند نیز استفاده کرد.
- برخی از زبان‌ها تنها یک پارادایم را پشتیبانی می‌کنند. برای مثال هسکل فقط برای برنامه‌نویسی تابعی به کار می‌رود.

تاریخچه زبان‌های برنامه‌نویسی

- در دهه ۱۹۵۰ تعدادی از زبان‌های برنامه‌نویسی برای تسهیل نوشتن دستورات کامپیوتری که تا قبل از آن به زبان اسمبلی نوشته می‌شدند به وجود آمدند. قبل از به وجود آمدن زبان‌های برنامه‌نویسی، به ازای هر نوع معماری سخت‌افزاری یک زبان اسمبلی وجود داشت. با این که زبان اسمبلی زبانی است که به زبان ماشین شباهت بیشتری دارد تا زبان انسان، و بنابراین نوشتن برنامه با استفاده از این زبان نسبتاً دشوار است اما به علت کنترلی که برنامه‌نویس بر روی سخت‌افزار دارد، با استفاده از آن برنامه‌های کارآمدی می‌توان نوشت.
- اولین زبان برنامه‌نویسی در یک رساله دکتری در سال ۱۹۵۱ توسط کورادو بوهم^۱ در دانشگاه ای‌تی‌اچ زوریخ توصیف و به همراه یک کامپایلر عرضه شد.
- دو زبان مهم تجاری که در این دهه به وجود آمدند، عبارتند از فورترن و کوپول.
- فورترن در بین سال‌های ۱۹۵۴ و ۱۹۵۶ توسط تیمی به رهبری جان باکوس^۲ در آی‌بی‌ام به وجود آمد.

^۱ Corrado Bohm

^۲ John Backus

- نوآوری جدید فورترن این بود که به برنامه نویسی کمک می‌کرد تا بتواند فرمول‌های ریاضی را به همان صورتی که بر روی کاغذ نوشته می‌شود بنویسد. در واقع کلمه فورترن مخفف کلمه ترجمه فرمول¹ بود. برای مثال برنامه‌نویسان فورترن می‌توانستند فرمولی مانند $i + 2 * j$ را بنویسند. تا قبل از آن نیاز بود که برنامه نویسی متغیر i را در یک رجیستر ذخیره کند و j را در یک رجیستر دیگر. سپس j را دو برابر کند و سپس مقدار این دو رجیستر را با هم جمع کند. بنابراین فورترن به برنامه‌نویسان کمک می‌کرد که فرمول‌های ریاضی را به زبان خود بنویسند و نه به زبان کامپیوتر و کامپایلر عملیات مورد نیاز برای تبدیل فرمول به زبان اسمبلی را انجام می‌داد.

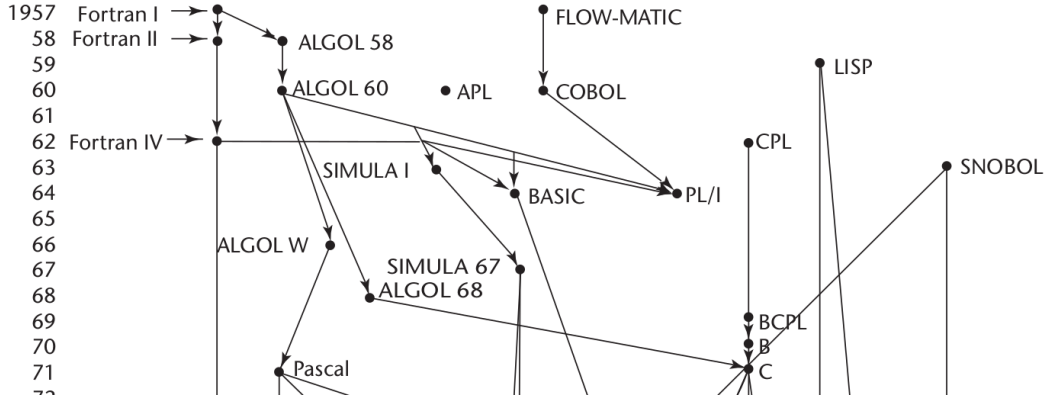
¹ Formula Translation

- فورترن همچنین دارای زیربرنامه و همچنین آرایه بود و بدین ترتیب برنامه‌نویسان می‌توانستند برنامه‌های قابل فهم‌تری بنویسند. البته فورترن دارای محدودیت‌هایی نیز بود. به طور مثال با استفاده از فورترن یک تابع نمی‌توانست خود را فراخوانی کند، زیرا برای این کار به تکنیک‌هایی نیاز بود که تا آن زمان به وجود نیامده بودند.
- در همان دوره زبان کوبول نیز برای استفاده در برنامه‌های تجاری توسط گریس هاپر¹ به وجود آمد. دستورات کوبول به زبان انسان شباهت زیادی داشت.

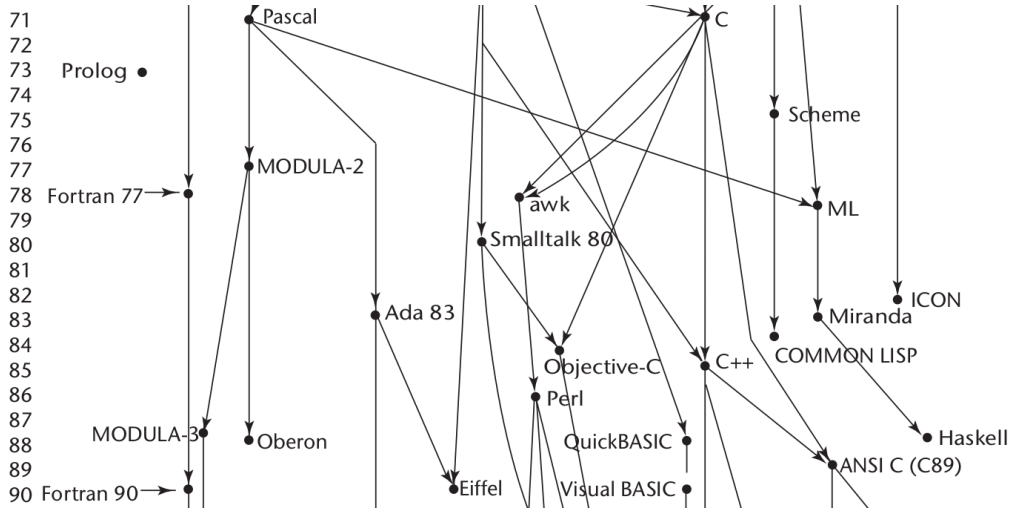
¹ Grace M. Hopper

- در اواخر دهه ۱۹۵۰ و اوایل دهه ۱۹۶۰ زبان‌های الگول و لیسپ به وجود آمدند. در این زبان‌ها امکان فراخوانی تابع توسط خودش و بنابراین نوشتن توابع بازگشتی وجود داشت.
- در دهه ۱۹۷۰ روش‌هایی برای ساختاربندی داده‌ها و ایجاد نوع داده‌های انتزاعی به وجود آمدند. با استفاده از این روش‌ها برنامه‌های پیچیده نظم بیشتری پیدا می‌کردند.
- با افزایش سرعت سخت‌افزار به تکنیک‌هایی برای استفاده بهینه از آنها نیاز بود و بنابراین برنامه‌نویسی همروند برای اجرای قسمت‌های مختلف یک برنامه به طور همزمان و یا اجرای چند برنامه به طور همزمان به وجود آمد. همچنین با به وجود آمدن شبکه‌های کامپیوتری به روش‌هایی برای بهره‌گیری از چندین کامپیوتر به طور همزمان نیاز بود و بنابراین برنامه‌نویسی توزیع شده به وجود آمد.

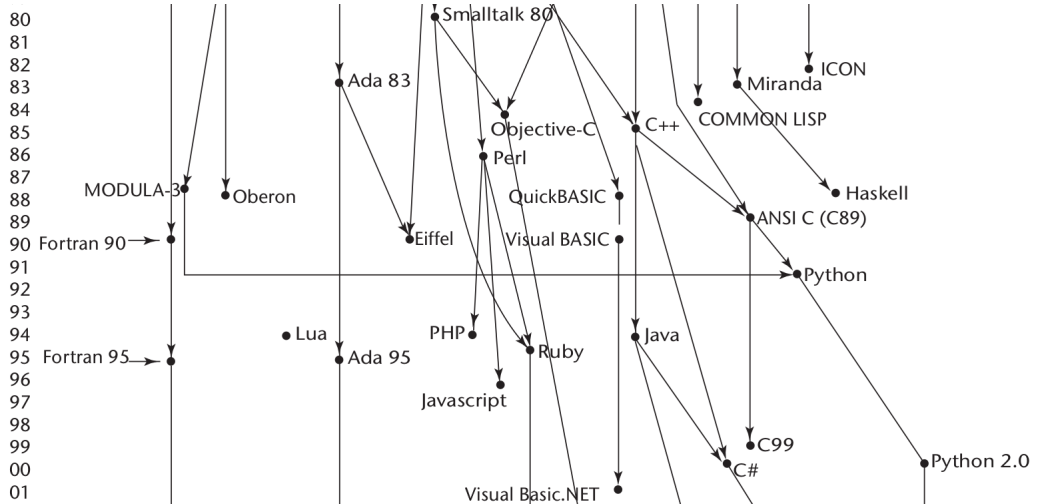
تاریخچه زبان‌های برنامه‌نویسی



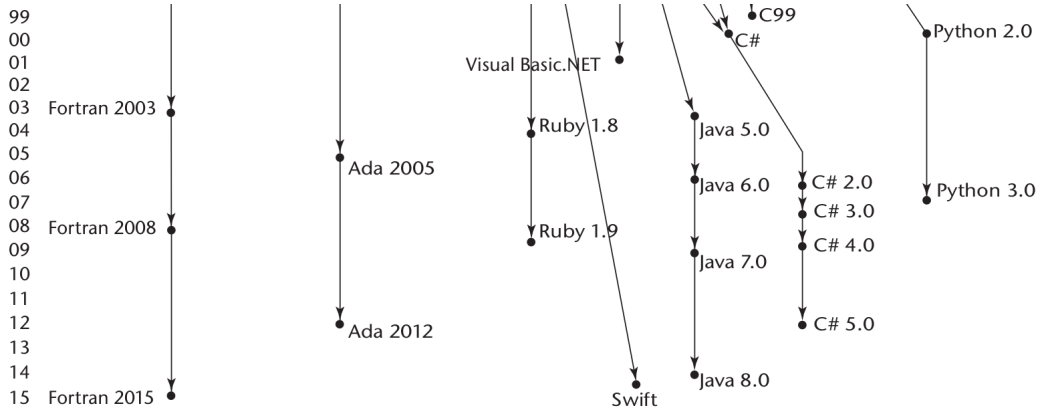
تاریخچه زبان‌های برنامه‌نویسی



تاریخچه زبان‌های برنامه‌نویسی



تاریخچه زبان‌های برنامه‌نویسی



- زبان‌های برنامه‌نویسی می‌توانند به یکی از سه روش زیر پیاده‌سازی شوند : ترجمه¹، تفسیر²، پیاده‌سازی ترکیبی³

¹ compilation

² interpretation

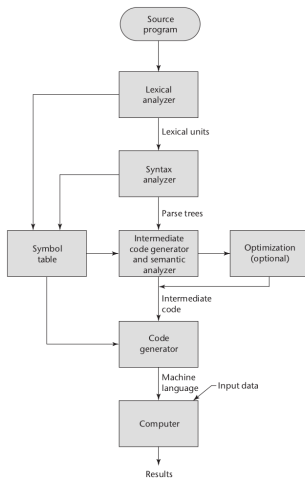
³ hybrid implementation

- یک برنامه کامپیوتری می‌تواند مستقیماً به زبان ماشین ترجمه شده، و بر روی کامپیوتر مقصد اجرا شود. این روش را پیاده سازی توسط کامپایلر¹ یا مترجم می‌نامیم.
- مزیت این روش این است که برنامه پس از کامپایل شدن با سرعت بالایی اجرا می‌شود.
- پیاده‌سازی زبان‌هایی مانند کوبول، سی و سی++ با استفاده از کامپایلر صورت گرفته است.
- زبانی را که یک کامپایلر ترجمه می‌کند زبان مبدأ² یا زبان منبع نامیده می‌شود.

¹ compiler

² source language

- در شکل زیر مراحل انجام ترجمه نشان داده شده است.



- تحلیل‌گر لغوی¹ متن نوشته شده در برنامه منبع² را به واحدهای لغوی یا واژگانی تبدیل می‌کند. این واحدهای لغوی که توکن نامیده می‌شوند می‌توانند شناسه‌ها³، کلمات کلیدی⁴، عملگرها⁵ و علائم نشانه گذاری⁶ باشند. تحلیل‌گر واژگانی، از توضیحات⁷ چشم پوشی می‌کند.

¹ lexical analyzer

² source program

³ identifier

⁴ key word

⁵ operator

⁶ punctuation symbol

⁷ comment

- تحلیل‌گر نحوی¹ توکن را از تحلیل‌گر لغوی دریافت می‌کند و با استفاده از آنها خطاهای نحوی برنامه را تشخیص می‌دهد و همچنین یک ساختار به نام درخت تجزیه² می‌سازد. این درخت‌های تجزیه ساختار نحوی یک برنامه را نشان می‌دهند.
- تولیدکننده کد واسط³ یک برنامه به یک زبان دیگر تولید می‌کند که حد واسط برنامه منبع و برنامه زبان ماشین⁴ است. این زبان میانی معمولاً کمی انتزاعی‌تر از زبان اسمبلی است.

¹ syntax analyzer

² parse tree

³ intermediate code generator

⁴ machine language

- تحلیل‌گر معنایی¹ که بخشی از تولید کننده کد میانی یا کد واسط است، خطاهای معنایی برنامه را بررسی می‌کند.
- کد تولید شده توسط تولید کننده کد میانی، در برخی موارد به یک واحد بهینه سازی² داده می‌شود تا کد تولید شده را در صورت امکان کوچک‌تر و سریع‌تر کند. بسیاری از بهینه سازی‌ها را به سختی می‌توان بر روی کد زبان ماشین انجام داد بنابراین این بهینه سازی‌ها بر روی کد واسط انجام می‌شوند.
- در نهایت تولید کننده کد³، کد بهینه سازی شده در زبان میانی را به برنامه‌ای در زبان ماشین ترجمه می‌کند.

¹ semantic analyzer

² optimization

³ code generator

- جدول علائم¹ اطلاعات مورد نیاز برای فرایند کامپایل را نگهداری می‌کند. محتوای این جدول همه نمادها و نوع آنهاست که برای فرایند تولید کد مورد نیاز است. این گونه اطلاعات توسط تحلیل‌گر لغوی و نحوی در جدولی نگهداری می‌شوند و توسط تحلیل‌گر معنایی و تولیدکننده کد استفاده می‌شوند.
- دقت کنید که برنامه‌ی نوشته شده تقریباً هیچگاه به تنهایی قابل استفاده نیست بلکه معمولاً نیاز به برنامه‌های جانبی است که توسط برنامه‌های دیگر یا سیستم عامل نوشته شده‌اند. برای مثال برای استفاده از ورودی و خروجی، برنامه نیاز به برنامه‌های جانبی دارد که توسط سیستم عامل برای استفاده از ورودی و خروجی مهیا شده‌اند.

¹ symbol table

- بنابراین قبل از این که کد تولید شده به زبان ماشین بتواند اجرا شود، برنامه‌های جانبی باید به برنامه مورد نظر پیوند¹ داده شوند. در این فرایند پیوند دادن یا لینک کردن، برنامه‌های مورد نیاز دیگر (که توسط برنامه نویسان دیگر پیاده سازی شده‌اند) و یا برنامه‌های سیستمی (که توسط سیستم عامل مهیا شده‌اند) باید به کد ماشین تولید شده لینک شوند. برای این کار آدرس کد ماشین برنامه‌های جانبی به برنامه مورد نظر برای اجرا داده می‌شود.
- فرایند پیوند کدهای جانبی به کد تولید شده، توسط یک پیوند دهنده² یا لینکر انجام می‌شود.
- بنابراین لینکر وظیفه دارد کد ماشین تولید شده را به کد ماشین برنامه‌های جانبی که به صورت کتابخانه‌ها با ارائه توابع پر استفاده توسعه داده شده‌اند و کد ماشین برنامه‌های سیستمی که توسط سیستم عامل مهیا شده‌اند، پیوند دهد.

¹ Link

² Linker

- یک پیش پردازنده¹ برنامه‌ای است که یک برنامه را قبل از کامپایل شدن پردازش می‌کند. در مرحله پیش پردازش، معمولاً دستوراتی که میانبر برای دستورات دیگر هستند حذف می‌شوند و با دستورات اصلی جایگزین می‌شوند.
- برای مثال در زبان سی با استفاده از دستور `#include "lib.h"` محتوای فایل `lib.h` در ابتدای برنامه قرار می‌گیرد. در مرحله پیش پردازش محتوای فایل مورد نظر به محتوای فایل کد منبع الحاق می‌شود.

¹ preprocessor

- به عنوان مثال دیگر، در زبان سی می‌توانیم با استفاده از دستور `#define` نام‌های نمادین ایجاد کنیم. به قواعد و دستوراتی که مشخص می‌کنند چگونه یک الگوی خروجی بر اساس یک الگوی ورودی تولید شود، ماکرو¹ گفته می‌شود.
- برای محاسبه ماکزیمم دو عدد می‌توانیم ماکرویی به صورت زیر تعریف کنیم.

$$\text{\#define max(A,B) ((A)>(B) ? (A):(B))}$$
 حال اگر در برنامه منبع داشته باشیم

$$\text{max(x+2,y)}$$
 در مرحله پیش پردازش این دستور به دستور $(x+2):(y) ? (x+2)>y$ تبدیل می‌شود.
- در پیاده‌سازی زبان به روش ترجمه، سرعت انتقال کد از حافظه به پردازنده معمولاً کمتر از سرعت اجرای کد در پردازنده است و بنابراین سرعت اجرای برنامه را سرعت انتقال کد تعیین می‌کند. به این پدیده تنگنای معماری وان نویمان² می‌گوییم.

¹ macro

² Van Neumann bottleneck

- تفسیر روش دیگری برای پیاده سازی زبان برنامه نویسی است. با استفاده از این روش، برنامه های زبان منبع توسط یک مفسر¹ خط به خط خوانده می شوند، به زبان ماشین تبدیل شده و اجرا می شوند.
- مزیت این روش این است که کد برنامه نیاز به کامپایل ندارد و همه جا قابل استفاده است. همچنین با سرعت بیشتری می توان برنامه های کامپیوتری تولید کرد چرا که کد برنامه را می توان به راحتی با استفاده از مفسر خط به خط اجرا و تست نمود.
- از طرفی دیگر عیب این روش این است که معمولاً زمان اجرای برنامه ها در آن نسبت به روش ترجمه پایین تر است.

¹ interpreter

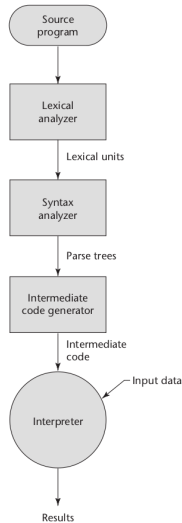
- در روش تفسیر تنگنای زمان اجرا، کدگشایی دستورات است که بسیار زمان‌بر است و نه انتقال دستورات از حافظه به پردازنده.
- زبان‌هایی مانند ای‌پی‌ال¹ و لیسپ از زبان‌های دههٔ ۱۹۶۰ توسط مفسر پیاده‌سازی شدند. در سال‌های اخیر زبان‌های وب مانند پی‌اچ‌پی و جاوااسکریپت نیز توسط مفسر پیاده‌سازی می‌شوند.

¹ APL

- برخی از زبان‌های برنامه‌نویسی توسط یک روش ترکیبی بین تفسیر و ترجمه پیاده سازی شده‌اند. در این نوع پیاده سازی، زبان سطح بالا ابتدا به یک برنامه در یک زبان میانی ترجمه می‌شود. سپس تفسیر از زبان میانی به زبان ماشین آسان‌تر می‌تواند اجرا شود.

پیاده سازی ترکیبی

- روند پیاده سازی زبان در این روش در شکل زیر نشان داده شده است.



- زبان پرل¹ و پایتون² به این روش پیاده سازی شده‌اند.
- همچنین زبان جاوا به این روش پیاده سازی شده است. زبان میانی بایت کد³ (کد بایتی) نامیده می‌شود. با استفاده از این روش می‌توان بایت کد را بر روی هر سیستمی که دارای ماشین مجازی جاوا باشد اجرا نمود.
- زبان‌های برنامه‌نویسی به روش‌های مختلفی پیاده سازی شده‌اند. با استفاده از مترجم، سرعت اجرای برنامه بهبود پیدا می‌کند. پیاده‌سازی توسط مفسر برای قابلیت جابجایی برنامه منبع و همچنین استفاده و خطایابی آسان‌تر و پیاده‌سازی سریع‌تر توسط برنامه نویس ارائه می‌شود. یک پیاده سازی ترکیبی برای فراهم کردن امکان جابجایی کد و سرعت اجرای بالاتر نسبت به مفسر استفاده می‌شود.

¹ Perl

² Python

³ byte code